

DevOps Interview Preparation Series (Day 5)

 by T Keerti

Terraform

1)What is Terraform and why is it used in DevOps?

Ans: Terraform is an open-source **Infrastructure as Code (IaC)** tool by HashiCorp used to **provision, manage, and version** cloud infrastructure (AWS, Azure, GCP, etc.). It helps automate cloud setup and maintain consistency across environments using declarative configuration files (.tf).

2)What language does Terraform use?

Ans: Terraform uses **HCL (HashiCorp Configuration Language)** — human-readable and JSON-compatible for defining infrastructure resources.

3)How is Terraform different from CloudFormation?

Feature	Terraform	CloudFormation
Cloud	Multi-cloud	AWS-only
Language	HCL	JSON/YAML
Modules	Supported	Supported
State Mgmt	Local/Remote	Managed by AWS
Speed	Faster	Slower on large stacks

4) What are the main Terraform commands and workflow

terraform init → Initializes provider plugins
terraform validate → Validates syntax
terraform plan → Previews what will be created/changed
terraform apply → Applies infrastructure changes
terraform destroy → Destroys resources

5)Explain the "desired vs actual state" concept

Terraform compares the **desired state** (from .tf files) with the **actual state** (recorded in the state file) and applies only required changes to bring them in sync.

6)What is Idempotency in Terraform?

Ans: Running terraform apply multiple times will not recreate unchanged resources — this is **idempotency**

7)What are Providers in Terraform?

Providers are **plugins** that allow Terraform to interact with cloud APIs (e.g., aws, azure, google).
Example:

```
provider "aws" {  
  region = "us-east-1"  
}
```

8)How to lock provider versions?

```
terraform {  
  required_providers {  
    aws = {  
      source  = "hashicorp/aws"  
      version = "~> 5.0"  
    }  
  }  
}
```

9). What are Resources in Terraform?

Resources are **infrastructure components** (e.g., EC2, S3, VPC).

```
resource "aws_instance" "web" {  
  ami = "ami-123456"  
  instance_type = "t2.micro"  
}
```

10)What are Data Sources?

Data sources **fetch existing information** instead of creating new resources.

```
data "aws_ami" "latest" {  
  owners = ["amazon"]  
  most_recent = true  
  filter {  
    name = "name"  
    values = ["amzn2-ami-hvm*"]  
  }  
}
```

11)What is the difference between Resource and Data?

Feature	Resource	Data
Action	Creates/Manages infra	Reads existing info
Example	<code>aws_instance</code>	<code>aws_ami</code>

12) What is Terraform State?

Terraform stores metadata (like resource IDs, attributes) in a **state file** (terraform.tfstate). It acts as the **single source of truth** for Terraform's current infrastructure.

13) What are the types of state backends?

Local — stores in your system (terraform.tfstate)

Remote — stores in cloud storage (e.g., **S3**, **Azure Blob**) Enables team collaboration,Locks state to prevent race conditions,Centralizes infra tracking

Example:

```
terraform {  
  backend "s3" {  
    bucket = "my-terraform-state"  
    key    = "prod/terraform.tfstate"  
    region = "us-east-1"  
  }  
}
```

14) What is State Locking?

Prevents multiple users from modifying infra simultaneously.

Example: S3 backend for AWS.

16)How to inspect state?

terraform show — shows current state

terraform state list — lists all resources

terraform state show <resource> — details one resource

17)What are Variables in Terraform?

Variables make configuration **reusable and dynamic**.

```
variable "region" {  
  type    = string  
  default = "us-east-1"  
}
```

18). How to pass variables?

Inline: terraform apply -var="region=us-west-2"

File: terraform apply -var-file=dev.tfvars

Environment: export TF_VAR_region=us-east-1

19)What are Output Values?

Outputs display resource info after apply:

```
output "instance_ip" {  
  value = aws_instance.web.public_ip  
}
```

Useful for pipelines or chaining stacks.

20)What is Terraform.tfvars?

Stores variable values separately

```
region = "us-west-1"  
instance_type = "t3.micro"
```

21)How to create an AWS EC2 instance using Terraform?

```
provider "aws" {  
  region = "us-east-1"  
}  
  
resource "aws_instance" "example" {  
  ami           = "ami-0c55b159cbfafe1f0"  
  instance_type = "t2.micro"  
  tags = {  
    Name = "TerraformEC2"  
  }  
}
```

22)How to create a VPC, subnet, and EC2 in one configuration?

```
resource "aws_vpc" "main" {  
  cidr_block = "10.0.0.0/16"  
}  
  
resource "aws_subnet" "sub1" {  
  vpc_id     = aws_vpc.main.id  
  cidr_block = "10.0.1.0/24"  
}  
  
resource "aws_instance" "web" {  
  ami           = "ami-123456"  
  instance_type = "t2.micro"  
  subnet_id     = aws_subnet.sub1.id  
}
```

23)How to attach IAM role to EC2 in Terraform?

```
resource "aws_iam_role" "ec2_role" {  
  name = "ec2_role"  
  assume_role_policy = jsonencode({  
    Version = "2012-10-17"  
    Statement = [{  
      Action = "sts:AssumeRole"  
      Effect = "Allow"  
      Principal = { Service = "ec2.amazonaws.com" }  
    }]  
  })  
}
```

24)How to manage S3 bucket using Terraform?

```
resource "aws_s3_bucket" "bucket" {  
  bucket = "my-tf-bucket"  
  acl    = "private"  
  versioning {  
    enabled = true  
  }  
}
```

25)How to enable S3 logging?

```
logging {  
  target_bucket = aws_s3_bucket.log_bucket.id  
  target_prefix = "log/"  
}
```

26)How to reference one resource in another?

Using implicit dependency via attributes:

```
subnet_id = aws_subnet.main.id
```

27)What is the depends_on argument used for?

For **explicit dependencies**:

```
resource "aws_instance" "web" {  
  depends_on = [aws_security_group.sg]  
}
```

28)How to tag all AWS resources consistently?

Use a locals block

```
locals {  
  common_tags = {  
    Environment = "dev"  
    Owner       = "Keerti"  
  }  
}
```

tags = local.common_tags

29)How to configure AWS credentials securely for Terraform?

Options:AWS CLI profile,Environment variables,Shared credentials file

Avoid hardcoding in .tf files.

30)How to create an AWS security group using Terraform?

```
resource "aws_security_group" "web_sg" {  
  vpc_id = aws_vpc.main.id  
  ingress {  
    from_port = 22  
    to_port   = 22  
    protocol  = "tcp"  
    cidr_blocks = ["0.0.0.0/0"]  
  }  
  egress {  
    from_port = 0  
    to_port   = 0  
    protocol  = "1"  
    cidr_blocks = ["0.0.0.0/0"]  
  }  
}
```

31)How do you import an existing AWS resource into Terraform?

```
terraform import aws_instance.web i-0123456789abcdef
```

32)How to ignore changes to specific attributes?

```
lifecycle {  
  ignore_changes = [tags]  
}
```

33)How to prevent resource deletion by mistake?

```
lifecycle {  
  prevent_destroy = true  
}
```

34)How to force resource recreation?

terraform taint <resource>

→ Marks it for recreation on next apply.

Use terraform apply -replace=aws_instance.web in newer versions.

35)How to count resources dynamically?

```
resource "aws_instance" "web" {  
  count = 3  
  ami   = "ami-123"  
  instance_type = "t2.micro"  
}
```

36). How to create multiple resources with different names?

for_each = toset(["dev", "prod"])

37)How to store secrets securely in Terraform? Use AWS Secrets Manager

38)How to upgrade Terraform provider plugins? terraform init -upgrade

39) . What is a Terraform refresh?

terraform refresh updates the **state file** to match **real-world infrastructure**.

40)What are Terraform Modules?

module is a container for multiple resources that are used together. It helps reuse and organize infrastructure.
Example:

```
module "vpc" {  
  source = "../modules/vpc"  
  cidr_block = "10.0.0.0/16"  
}
```

Benefits:

- Code reusability
- Better organization
- Easier collaboration
- Consistency across environments

Real-time use case:

Commonly used for reusable setups like network, iam, eks_cluster, or ec2_instance.

41)What is a Root Module and Child Module?

Root module: The main directory where you run Terraform commands.

Child module: Modules that are called from the root module.

This structure allows separation of environments like dev, stage, prod.

42)What is a Terraform Workspace?

Used to manage multiple environments (dev, stage, prod) using the same codebase

```
terraform workspace new dev  
terraform workspace list  
terraform workspace select prod
```

43)How do you handle sensitive data in Terraform?

Use sensitive = true for variable values
Store secrets in **AWS Secrets Manager** / **Azure Key Vault**
Never commit .tfstate or .tfvars to Git
Use **Terraform Cloud Vault integration**

```
variable "db_password" {  
  type = string  
  sensitive = true  
}
```

44)) How do you integrate Terraform in CI/CD pipelines (GitHub Actions / GitLab / Jenkins

```
terraform init -check  
terraform validate  
terraform plan -out=planfile  
Manual approval  
terraform apply planfile
```

45)How to perform blue-green deployment using Terraform?

Create two sets of resources (Blue & Green) and switch traffic using Route53 or Load Balancer.

46)How to connect Terraform-managed EC2 with RDS securely?

Create RDS in private subnet

Attach EC2 in public subnet with SG allowing DB port 3306 to RDS SG only

Use variables for DB endpoint from Terraform output.

47)How to handle cross-region resource creation?

Same as multiple providers, but region changes dynamically.

48)How to provision EKS using Terraform?

```
module "eks" {  
  source = "terraform-aws-modules/eks/aws"  
  cluster_name = "dev-eks"  
  vpc_id       = aws_vpc.main.id  
  subnet_ids   = [aws_subnet.public1.id, aws_subnet.public2.id]  
}
```

49)Explain Terraform standard directory structure

```
project/  
├── main.tf  
├── variables.tf  
├── outputs.tf  
├── provider.tf  
├── terraform.tfvars  
├── modules/  
│   ├── vpc/  
│   │   ├── main.tf  
│   │   ├── variables.tf  
│   │   └── outputs.tf
```

50)What is terraform fmt and terraform validate?

terraform fmt → Formats the code

terraform validate → Checks syntax and logical correctness

51)Terraform Apply Fails – How do you debug?

Check .terraform directory

Run TF_LOG=DEBUG terraform apply

Check AWS IAM permissions

Validate provider version mismatches

52)Difference between count and for_each

count → key-based iteration

for_each → integer-based iteration (better for maps/sets)

53)Terraform vs Ansible

Feature	Terraform	Ansible
Type	IaC (Provisioning)	Configuration Management
Language	HCL	YAML
Execution	Declarative	Procedural
Idempotency	Yes	Yes
Use Case	Build infra	Configure infra

54)What are Dynamic Blocks in Terraform

Used when repeating nested arguments dynamically.

```
dynamic "ingress" {  
  for_each = var.rules  
  content {  
    from_port = ingress.value.from  
    to_port   = ingress.value.to  
  }  
}
```

55)What are Lifecycle Hooks in Terraform?

Control resource creation/destruction order:

```
create_before_destroy = true  
prevent_destroy       = true  
ignore_changes        = [tags]
```

56)What is the difference between terraform refresh and terraform plan?

- refresh updates state file
- plan compares desired vs actual state and shows changes